

```
//LIBRERÍAS
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

//CONSTANTES
#define RETARDO_REBOTES 200
#define PIN_RED 18
#define PIN_GREEN 19
#define BUZZER 14
#define BUTTON1 27
#define BUTTON2 16
#define BUTTON3 17
#define RELE 4
#define SCREEN_ADDRESS 0x3C // I2C screen address

//VARIABLES
Adafruit_SSD1306 display(128, 64, &Wire, -1); //128x64 screen
bool bombaAExplotar = false; //Creamos una variable
//booleana para cada uno de los 4 posibles estados
bool bombaExplotada = false;
bool bombaDesactivada = false;
bool cuentaEnMarcha = false;
int segundos = 4; // segundos con los que va a comenzar
//la cuenta atrás
int decimas = 5; //decimas con los que va a comenzar
int centesimas = 5;

//FUNCIONES
//Hace sonar un pitido con una cierta duración en ms y una nota del 0 al
100
//Precisa estas instrucciones en el SETUP:
//ledcSetup(0, 0, 10); // canal, frecuencia, resolucion en bits
//ledcAttachPin(BUZZER, 0); // se asigna el pin del buzzer y el canal
void beep(float duracion, unsigned int nota)
{
    int frecuencia;
    if (nota < 100)
    {
        //Calculamos la frecuencia a tocar, indice 0 Do4
        //esto no son más que cálculos para no tener que escribir la nota
```

```

musical con la frecuencia exacta
    //Más info en
https://www.pinguytaz.net/index.php/2021/09/21/musica-con-un-arduinoesp32-y-un-buzzer/
    frecuencia = (int) 261.625 * pow(1.0594630943593, nota);
}
else frecuencia = 0;

    ledcWriteTone(0, frecuencia); //Toca Tono (PWM) en el canal marcado a
la frecuencia indicada
    delay(duracion);              //Con este delay marco la duración del tono
    ledcWrite(0, 0);              //Poniendo en el canal 0 un duty de 0 lo
para
}
//Hace sonar una cierta melodía de inicio usando la función beep creada
antes
void melodiaInicio()
{
    beep(200, 5); //frecuencia y nota
    beep(200, 6);
    beep(200, 7);
    beep(200, 8);
    beep(200, 9);
}
//Lee todos los pulsadores y asigna estados
void lecturaPulsadores()
{
    //Esto implica la puesta en marcha de la cuenta; sólo si no estaba ya
en marcha
    if (digitalRead(BUTTON1) == 0 && !cuentaEnMarcha) //control de
activacion de la cuenta
    {
        cuentaEnMarcha = true; //activo la cuenta atrás
        delay(RETARDO_REBOTES); //quito rebotes para no darle al B1 justo
después de activar, lo que provocaría explosión
    }
    //Esto implica que estoy pulsando el botón incorrecto para desactivar
la bomba, durante la cuenta, y no estando la bomba previamente
desactivada
    if ( (digitalRead(BUTTON1) == 0 || digitalRead(BUTTON2) == 0) &&
cuentaEnMarcha && !bombaDesactivada) //control de estado para explotar
    {
        bombaAExplotar = true; //la bomba va a explotar en cuanto se
compruebe que este estado está activo
        cuentaEnMarcha = false; //paramos la cuenta atrás
    }
}

```

```

    //Esto implica que desactivo la bomba, si la cuenta estaba activa y no
    había sido previamente desactivada ni explotada
    if (digitalRead(BUTTON3) == 0 && cuentaEnMarcha && !bombaDesactivada
    && !bombaExplotada) //control de estado de desactivación
    {
        bombaDesactivada = true; //desactivo bomba
    }
}

//Funcion muy simple para hacer una cuenta atras: cada 5 ms (en vez de
cada 10ms) de segundo se decrementa el contador de centésimas. Buscamos
darle más realismo a la cuenta atrás
void cuentaAtras()
{
    if (cuentaEnMarcha && !bombaExplotada && !bombaAExplotar &&
    !bombaDesactivada) // Con esto controlamos la cuenta atras que solo se
    activara cuando la bomba no ha explotado ni esta por explotar ni esta
    desactivada
    {
        if ( centesimas > 0)    //esto es para que las centésimas no sigan
        contando más allá de 0 hacia números negativos
        {
            centesimas--;
            delay(5); //al meter un retardo de 5ms, estoy actuando sobre
            esta cuenta cada centesima de segundo;
            //no es preciso porque no tiene en cuenta el tiempo de ejecución
            de las instrucciones, pero nos vale
        }
        if ( centesimas == 0 && decimas > 0)    //cuando las centésimas
        sean 0, y las décimas mayores que 0, las décimas seguirán decreciendo
        desde el nr 5 hasta el 0
        {
            decimas--;
            centesimas = 5;
        }
        if (centesimas == 0 && decimas == 0 && segundos > 0)
        ///cuando las centésimas y las décimas sean 0, y los segundos mayor que
        0, los segundos decrecerán desde 4 que hemos establecido antes hasta 0.
        Luego con los segundos a 0, las décimas y las centésimas seguirán
        decreciendo desde el nr 5 hasta el 0
        {
            segundos--;
            decimas = 5;
            centesimas = 5;
        }
        display.clearDisplay();
    }
}

```

//esto es para configurar qué

se verá en la pantalla principal

```
    display.setCursor(0, 0);
    display.setTextSize(1);
    display.printf("Pulsa B3 y desactiva");
    display.setTextSize(2);
    display.setCursor(15, 25);
    display.print(0);
    display.print(segundos);
    display.printf(":");
    display.print(decimas);
    display.print(centesimas);
    display.display();
}
}
//Controla si la bomba ha sido desactivada y muestra por pantalla lo que
corresponda
void chequearBomba()    //esta función la creamos porque nos va a ser
fácil llamarla simplemente en el loop. Es la tercera función que
llamaremos al final. Consistirá de lo siguiente:
{
    if (bombaDesactivada) // con esto controlamos la desactivación de la
bomba
    {
        display.clearDisplay();
        display.setCursor(0, 10);
        display.setTextSize(2);
        display.printf("  BOMBA  DESACTIVA");
        display.display();
        digitalWrite(PIN_GREEN, 1);
        delay(500);
        digitalWrite(PIN_GREEN, 0);
        digitalWrite(PIN_RED, 1);
        delay(500);
        digitalWrite(PIN_RED, 0);
    }
    if (bombaAExplotar) // con esto controlamos la explosión de la bomba
    {
        explosion();
    }
}
//Dibuja la pantalla en blanco y actualiza estado de la bomba
void explosion()          //dentro de la función "bombaAExplotar"
estamos creando el qué va a hacer la función "Explosión": pantalla en
blanco, el sonido final, y la activación de relé
{
    display.clearDisplay();
```

```

display.drawRect(0, 0, 128, 64, SSD1306_WHITE);
display.fillRect(0, 0, 128, 64, SSD1306_WHITE);
display.display();
if (!bombaExplotada) {
    beep(1000, 1);
}
digitalWrite(RELE, 1);
bombaExplotada = true;           //esta función se nombra solo al
//principio. ¿Por qué es necesario usarla aquí, de esta manera?
}

//INICIALIZACIÓN
void setup()
{
    //Establecemos los botones con resistencias internas de pullup
    pinMode(BUTTON1, INPUT_PULLUP);
    pinMode(BUTTON2, INPUT_PULLUP);
    pinMode(BUTTON3, INPUT_PULLUP);
    //Establecemos los pines del led bicolor
    pinMode(PIN_RED, OUTPUT);
    pinMode(PIN_GREEN, OUTPUT);
    //Establecemos el pin del relé (que está conectado a la bomba)
    pinMode(RELE, OUTPUT);
    //Preparamos la pantalla OLED
    display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS);
    display.clearDisplay();
    display.setTextSize(2);
    display.setTextColor(SSD1306_WHITE);
    display.display();
    //Limpiamos la pantalla, sacamos mensaje de bienvenida y hacemos pitar
    //el buzzer
    display.clearDisplay();
    display.setCursor(0, 5);
    display.printf("Bienvenido al juego explosivo");
    display.display();
    //Iniciamos valores del PWM con el que se genera el sonido
    ledcSetup(0, 0, 10);           // canal, frecuencia, resolucion en bits
    ledcAttachPin(BUZZER, 0);     // se asigna el pin del buzzer y el canal
    Serial.begin(115200);         // velocidad del puerto serie
    melodiaInicio();
}

// BUCLE PRINCIPAL. Simples llamadas a las funciones que hemos creado
void loop()
{
    lecturaPulsadores();
}

```

```
cuentaAtras();  
chequearBomba();  
}
```